Electron Momentum Transfer
User Manual

May 15, 2026

1 Compilation

The main program is compiled using CMake. It will automatically detect the host compiler and link the necessary libraries. To compile the C binaries from source, execute the following commands in your terminal:

```
git clone --depth 1 https://github.com/BanuDarius/electron-momentum-transfer
cd electron-momentum-transfer/
cmake -B build
cmake --build build
```

By default, CMake applies the `-march=native` and `-O3` optimization flags for your specific CPU architecture. If you are compiling on a legacy system or encounter illegal instruction errors, you can use CMake to build for a generic x86 architecture:

```
cmake -B build -DGENERIC=ON
cmake --build build
```

To delete the previously generated output data, including images and video renders, use the clean target:

```
cmake --build build --target clean-output
```

2 Quick functions

To easily check the functions of the program, you can uncomment one of the lines corresponding to these functions. To run a low resolution parameter sweep:

```
quick_example.run_quick_example(thread_num)
```

- `thread_num`: The number of threads to allocate for the simulation.

To reproduce one of the parameters sweeps presented in the `examples` directory:

```
examples.run_example(num_example, thread_num)
```

- `num_example`: The example to reproduce (1 to 4).
- `thread_num`: The number of threads.

To run a complete test between the numerical integrator and an analytical solution for one plane wave:

```
analytic_test.run_complete_test(thread_num)
```

- `thread_num`: The number of threads.

To replicate the results obtained by D. Bauer et al. (1995) Physical Review Letters paper:

```
examples.replicate_prl_results(thread_num)
```

- `thread_num`: The number of threads.

To run a strong scaling performance test, from 1 to N threads:

```
performance_test.run_example_performance_test(thread_num_final)
```

- `thread_num_final`: The maximum number of threads for the test.

3 Usage: `auto_compute.py`

The intended usage of this software suite is through the master Python script, `auto_compute.py`. This script serves as the primary user interface and handles the complete simulation.
To start full parameter sweep and auto-generate the analysis figures, simply execute:

```
python3 auto_compute.py
```

Configuration of the physical parameters (e.g., a_0, ω, thread count) should be performed directly within the variables defined at the top of the `auto_compute.py` script. The units used by the program are atomic units.

4 Laser Initialization and Parameters

In the `auto_compute.py` master script, the laser assembly is constructed by appending individual `LaserParameters` objects (defined in `sim_init.py`) to the `lasers` list.

```
lasers.append(sim_init.LaserParameters(a0, sigma, omega, etaf, zetax, zetay,
    alpha, phi, theta, psi, pond_integrate_steps))
```

The definitions of these arguments are:

- `a0` (a_0): The dimensionless potential amplitude $a_0 = \frac{|e|E_0}{mc\omega}$.
- `sigma` (σ): The width parameter of the Gaussian temporal envelope.
- `omega` (ω): The angular frequency of the laser field in atomic units.
- `etaf` (η_f): The flat-top envelope parameter. It defines the duration of the constant-amplitude plateau of the pulse in terms of the phase $\eta = \omega t - \mathbf{k} \cdot \mathbf{r}$. For a standard Gaussian pulse, set $\eta_f = 0$.
- `zetax`, `zetay` (ζ_x, ζ_y): The polarization parameters (ϵ_1 and ϵ_2). For instance, $\zeta_x = 0, \zeta_y = 1$ is for linear polarization, while $\zeta_x = \frac{1}{\sqrt{2}}, \zeta_y = \frac{1}{\sqrt{2}}$ is for circular polarization.
- `alpha` (α): The polarization rotation angle. It rotates the basis vectors ϵ_1 and ϵ_2 around the propagation axis $\mathbf{n}$ by the angle α.
- `phi`, `theta` (ϕ, θ): The azimuthal and polar spherical angles defining the laser's propagation direction vector $\mathbf{n}$.
- `psi` (ψ): The carrier-envelope phase (CEP), which shifts the phase of the oscillating carrier wave relative to the peak of the field envelope.

- `pond_integrate_steps`: A parameter used by the ponderomotive solver. It defines the number of steps used to evaluate the integrals (e.g., $\int A^2 d\phi$) with Simpson's 1/3 rule.

The line which runs the simulation itself is:

```
programs.run_simulation(method, sim_parameters, lasers)
```

- `method`: Defines the numerical integrator and physical approximation to be used. The Python string is internally mapped to a discrete `mode` integer passed to the C binary:
  - `"electromagnetic"` → `mode = 0`: Uses the Higuera-Cary particle pusher for the full Lorentz force equations.
  - `"ponderomotive"` → `mode = 1`: Uses the 4th-order Runge-Kutta (RK4) integrator to solve the averaged ponderomotive approximation.
  - `"electromagnetic-rk4"` → `mode = 2`: Uses the RK4 integrator for the Lorentz force (used for legacy comparisons or specific debugging).
- `sim_parameters`: An instance of the `SimParameters` class.
- `lasers`: The list of `LaserParameters` objects.

To perform data analysis efficiently without bottlenecking Python's memory space, the next script is used, which finds the final momentum for each particle:

```
programs.find_final_p(method, sim_parameters, axis_pos, axis_p)
```

- `method`: Specifies the source of the data (`"electromagnetic"`, `"ponderomotive"`, or `"analytic"`).
- `sim_parameters`: The current configuration state.
- `axis_pos`: The coordinate axis index (0 for X, 1 for Y, 2 for Z) corresponding to the initial spatial distribution of the particle assembly.
- `axis_p`: The coordinate axis index corresponding to the targeted final momentum component to be extracted.

The next program extracts the maximum momentum on one axis:

```
programs.find_max_p(method, sim_parameters, axis)
```

- `method`: `"electromagnetic"` or `"ponderomotive"`.
- `sim_parameters`: The current configuration state.
- `axis`: The coordinate axis index being analyzed.

5 Error Analysis and Visualization

The software is able to automatically benchmark the ponderomotive approximation against the full electromagnetic Lorentz force solver. This is executed with:

```
programs.calculate_errors(sim_parameters, axis)
```

- `sim_parameters`: The current configuration state.
- `axis`: The coordinate axis index being analyzed.

The average errors are plotted with:

```
plotting.plot_average_errors(a0_array, axis)
```

- `a0_array`: The linear space of a_0 values swept during the simulation.
- `axis`: The coordinate axis index corresponding to the momentum component.

For plotting the maximum momentum on one axis:

```
plotting.plot_max_p(method, a0_array, axis)
```

- `method`:`"electromagnetic"` or `"ponderomotive"`.
- `a0_array` : The linear space of a_0 values swept during the simulation (the X-axis of the plot).
- `axis`: The coordinate axis index corresponding to the momentum component being plotted (the Y-axis of the plot).

The heatmaps for visualizing the final momentum as a function of a_0:

```
plotting.plot_2d_heatmap_all(method, sim_parameters, a0_array, axis_pos, axis_p
    )
```

- `method`:`"electromagnetic"` or `"ponderomotive"`.
- `sim_parameters`: The current simulation state.
- `a0_array`: The linear space of a_0 values swept during the simulation.
- `axis_pos`: The coordinate axis index corresponding to the initial distribution of the particles (the X-axis of the heatmap).
- `axis_p` : The coordinate axis index corresponding to the final momentum component being mapped to the color gradient.

To plot the errors between the two methods:

```
plotting.plot_2d_errors_heatmap(sim_parameters, a0_array, axis_pos, axis_p)
```

- `sim_parameters`: The current simulation state.
- `a0_array`: The linear space of a_0 values swept during the simulation.
- `axis_pos`: The coordinate axis index corresponding to the initial position of the electrons.
- `axis_p`: The coordinate axis index corresponding to the final momentum component.

This function scans the trajectory data to identify when and when its momentum stabilizes (exits the field).

```
programs.find_enter_exit_time(method, sim_parameters, axis_pos, axis_p)
```

- `method`: `"electromagnetic"` or `"ponderomotive"`.
- `sim_parameters`: The current simulation state.
- `axis_pos`: The coordinate axis index representing the initial spatial position.

- `axis_p`: The coordinate axis index corresponding to the targeted momentum component.

This function automatically performs a numerical convergence test by re-running the simulation with a strictly higher temporal resolution and comparing the deviation in the final momentum.

```
programs.check_convergence(method, sim_parameters, lasers, axis_pos, axis_p,
    multiplier)
```

- `method`: The physics solver to validate.
- `sim_parameters`: The current simulation configuration.
- `lasers`: The list of laser parameters.
- `axis_pos`: The initial spatial position axis index.
- `axis_p`: The momentum axis index to check for convergence.
- `multiplier`: The factor by which to increase the number of integration steps.

To track the momentum of the particles as a function of time:

```
plotting.plot_time_momentum(method, sim_parameters, a0_array, axis_pos, axis_p)
```

- `method`: Specifies the physics solver utilized.
- `sim_parameters`: The simulation configuration.
- `a0_array`: The a_0 parameter sweep array.
- `axis_pos`: The initial spatial coordinate axis index.
- `axis_p`: The momentum component tracked on the Y-axis.

This function generates a phase-space diagram for each particle's momentum on the y axis and their positions on the x axis:

```
plotting.plot_phases(method, sim_parameters, a0_array, axis_pos, axis_p)
```

- `method`: Specifies the physics solver utilized.
- `sim_parameters`: The simulation configuration.
- `a0_array`: The a_0 parameter sweep array.
- `axis_pos`: The spatial coordinate mapped to the X-axis of the phase diagram.
- `axis_p`: The corresponding momentum component mapped to the Y-axis of the phase diagram.

This function generates a plot of the average convergence error as a function of a_0:

```
plotting.plot_convergence(method, a0_array, axis)
```

- `method`: Specifies the physics solver utilized.
- `a0_array`: The linear space of a_0 values swept during the simulation (mapped to the X-axis).

- `axis`: The coordinate axis index corresponding to the momentum component being analyzed for convergence.

To generate the complete convergence heatmap:

```
plotting.plot_2d_convergence_heatmap(method, sim_parameters, a0_array, axis_pos
    , axis_p)
```

- `method`: Specifies the physics solver being validated.
- `sim_parameters`: The current simulation configuration.
- `a0_array`: The linear space of a_0 values swept during the simulation (the Y-axis).
- `axis_pos`: The initial spatial coordinate axis index (the X-axis).
- `axis_p`: The momentum component axis index used to calculate the convergence.